

[开发指南](#)

OpenAI API 兼容

 Copy page

智谱AI 提供与 OpenAI API 兼容的接口，这意味着您可以使用现有的 OpenAI SDK 代码，只需要简单修改 API 密钥和基础 URL，就能无缝切换到 智谱AI 的模型服务。这种兼容性让您能够：

- 快速迁移现有的 OpenAI 应用
- 使用熟悉的开发模式和工具
- 享受智谱AI 模型的强大能力
- 保持代码的一致性和可维护性

 某些场景下智谱AI 与 OpenAI 接口仍存在差异，但不影响整体兼容性。

核心优势



零学习成本

如果您已经熟悉 OpenAI SDK，可以立即上手使用



快速迁移

现有 OpenAI 应用可以快速迁移到 智谱AI 平台



生态兼容

兼容 OpenAI 生态系统中的各种工具和框架



持续更新

跟随 OpenAI SDK 更新，保持最新功能支持

环境要求

>



Python 版本

Python 3.7.1 或更高版本



OpenAI SDK

OpenAI SDK 版本不低于 1.0.0

⚠️ 请确保使用 OpenAI SDK 1.0.0 或更高版本，旧版本可能存在兼容性问题。

安装 OpenAI SDK

使用 pip 安装

```
# 安装或升级到最新版本
pip install --upgrade 'openai>=1.0'

# 验证安装
python -c "import openai; print(openai.__version__)"
```

使用 poetry 安装

```
poetry add "openai>=1.0"
```

快速开始

获取 API Key

1. 访问 [智谱AI 开放平台](#)
2. 注册并登录您的账户

3. 在 **API Keys** 管理页面创建 API Key
4. 复制您的 API Key 以供使用



💡 建议将 API Key 设置为环境变量： `export ZAI_API_KEY=your-api-key` 替代硬编码到代码中，以提高安全性。

创建客户端

[基础配置](#) [环境变量](#) [配置类](#)

```
from openai import OpenAI

# 创建智谱AI 客户端
client = OpenAI(
    api_key="your-zhipuai-api-key",
    base_url="https://open.bigmodel.cn/api/paas/v4/"
)
```

基础使用示例

简单对话

```
client = OpenAI(
    api_key="your-zhipuai-api-key",
    base_url="https://open.bigmodel.cn/api/paas/v4/"
)

completion = client.chat.completions.create(
    model="glm-5.1",
    messages=[
        {"role": "system", "content": "你是一个聪明且富有创造力的小说作家"},
        {"role": "user", "content": "请你作为童话故事大王，写一篇短篇童话故事"}
    ],
    top_p=0.7,
    temperature=0.9
)

print(completion.choices[0].message.content)
```

流式响应

```
stream = client.chat.completions.create(
    model="glm-5.1",
    messages=[
        {"role": "user", "content": "写一首关于人工智能的诗"}
    ],
    stream=True,
    temperature=0.8
)

for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="", flush=True)

print() # 换行
```

>

```
class ChatBot:
    def __init__(self, api_key: str):
        self.client = OpenAI(
            > api_key=api_key,
            base_url="https://open.bigmodel.cn/api/paas/v4/"
        )
        self.conversation = [
            {"role": "system", "content": "你是一个有用的 AI 助手"}
        ]

    def chat(self, user_input: str) -> str:
        # 添加用户消息
        self.conversation.append({"role": "user", "content": user_input})

        # 调用 API
        response = self.client.chat.completions.create(
            model="glm-5.1",
            messages=self.conversation,
            temperature=1.0
        )

        # 获取 AI 回复
        ai_response = response.choices[0].message.content

        # 添加到对话历史
        self.conversation.append({"role": "assistant", "content": ai_response})

        return ai_response

    def clear_history(self):
        """清除对话历史，保留系统提示"""
        self.conversation = self.conversation[:1]

# 使用示例
bot = ChatBot("your-api-key")
print(bot.chat("你好，请介绍一下自己"))
print(bot.chat("你能帮我写代码吗？"))
print(bot.chat("写一个 Python 的快速排序算法"))
```

推理 (thinking)

在思考模式下¹，GLM-4.7 可以解决复杂的推理问题，包括数学、科学和逻辑问题。

```
import os
from openai import OpenAI

client = OpenAI(api_key='your-api-key', base_url='https://open.bigmodel.cn/api/paas/v1')
response = client.chat.completions.create(
    model='glm-4.7',
    messages=[
        {"role": "system", "content": "you are a helpful assistant"},
        {"role": "user", "content": "what is the revolution of llm?"}
    ],
    stream=True,
    extra_body={
        "thinking": {
            "type": "enabled",
        },
    }
)

for chunk in response:
    if chunk.choices[0].delta.reasoning_content:
        print(chunk.choices[0].delta.reasoning_content, end='')
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end='')
```

函数调用 (Function Calling)

```
def get_weather(location: str) -> str:
    """获取指定地点的天气信息"""
    # 这里应该调用真实的天气 API
    return f"{location} 的天气: 晴天, 温度 25°C"

# 定义函数描述
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "获取指定地点的天气信息",
            "parameters": {
                "type": "object",
                "properties": {
                    "location": {
                        "type": "string",
                        "description": "地点名称, 例如: 北京、上海"
                    }
                },
                "required": ["location"]
            }
        }
    }
]

# 调用带函数的对话
response = client.chat.completions.create(
    model="glm-5.1",
    messages=[
        {"role": "user", "content": "北京今天天气怎么样? "}
    ],
    tools=tools,
    tool_choice="auto"
)

# 处理函数调用
message = response.choices[0].message
```


BigModel

```
if message.tool_calls:
    for tool_call in message.tool_calls:
        if tool_call.function.name == "get_weather":
            > args = json.loads(tool_call.function.arguments)
            result = get_weather(args["location"])
            print(f"函数调用结果: {result}")
```

图像理解



```
import base64
from PIL import Image
```

```
import io
>
```

```
def encode_image(image_path: str) -> str:
```

```
    """将图像编码为 base64 字符串"""
```

```
    with open(image_path, "rb") as image_file:
```

```
        return base64.b64encode(image_file.read()).decode('utf-8')
```

```
# 图像理解示例
```

```
image_base64 = encode_image("path/to/your/image.jpg")
```

```
response = client.chat.completions.create(
```

```
    model="glm-4.6v",
```

```
    messages=[
```

```
        {
```

```
            "role": "user",
```

```
            "content": [
```

```
                {
```

```
                    "type": "text",
```

```
                    "text": "请描述这张图片的内容"
```

```
                },
```

```
                {
```

```
                    "type": "image_url",
```

```
                    "image_url": {
```

```
                        "url": f"data:image/jpeg;base64,{image_base64}"
```

```
                    }
```

```
                }
```

```
            ]
```

```
        }
```

```
    ],
```

```
    temperature=0.7
```

```
)
```

```
print(response.choices[0].message.content)
```

参数配置

参数	类型	默认值	说明
model	string	必填	要使用的模型名称
messages	array	必填	对话消息列表
temperature	float	0.6	控制输出的随机性 (0-1)
top_p	float	0.95	核采样参数 (0-1)
max_tokens	integer	-	最大输出 token 数
stream	boolean	false	是否使用流式输出
stop	string/array	-	停止生成的标记

⚠️ 注意：temperature 参数的区间为 (0,1)，do_sample = False (temperature = 0) 在 OpenAI 调用中并不适用。

实践建议

性能优化

- 使用连接池和会话复用
- 合理设置超时时间
- 实施异步调用处理高并发
- 缓存常用的响应结果

成本控制

- 合理设置 max_tokens 限制
- 使用合适的模型（不要过度使用强模型）
- 实施请求去重机制
- 监控 API 使用量

安全性

- 使用环境变量存储 API 密钥
- 实施输入验证和过滤
- 记录和监控 API 调用

可靠性

- 实施重试机制和错误处理
- 设置合理的超时时间
- 监控 API 状态和响应时间

迁移指南

从 OpenAI 迁移

如果您已经在使用 OpenAI API，迁移到智谱AI 非常简单：

```
# 原来的 OpenAI 代码
from openai import OpenAI

client = OpenAI(
    api_key="sk-...", # OpenAI API Key
    # base_url 使用默认值
)

# 迁移到智谱AI，只需要修改两个地方
client = OpenAI(
    api_key="your-zhipuai-api-key", # 替换为智谱AI API Key
    base_url="https://open.bigmodel.cn/api/paas/v4/" # 添加智谱AI base_url
)

# 其他代码保持不变
response = client.chat.completions.create(
    model="glm-5.1", # 使用智谱AI 模型
    messages=[{"role": "user", "content": "Hello!"}]
)
```

更多资源



智谱AI API 文档

查看智谱AI 完整的 API 文档



OpenAI 官方文档

参考 OpenAI 官方文档了解更多用法



BigModel

BigModel 致力于保持与 OpenAI API 的兼容性，如果您在迁移过程中遇到任何问题，请联系我们的技术支持团队。



>

< Claude API 兼容

LangChain 集成 >